本地多模态助手 · 源码阅读攻略

> 目标：不从头读 9000 行，而是**沿着"一次对话怎么走完"这条主线**把主干吃透。
> 全部行号以 **2026-11-10 的 `fb3d25a` 之后版本**为准；改了代码行号会漂，函数名不会。

---

0. 开始之前

环境

- 编辑器：PyCharm（`Ctrl+G` 跳行号，`Ctrl+Shift+F` 全局搜函数名）
- 建议同时开着程序跑：`run.py` 启动后浏览器访问 `http://127.0.0.1:8000`
- 边读边改一行 `console.log` / `print`，看输出对得上，比干读快三倍

一个判断技巧

项目里所有 ⚠️ 开头、或带 `2026-11-09` / `2026-11-10` 日期的注释，**都是踩过坑之后补的**，信息密度最高。遇到就细看。

第一遍 · 一次对话怎么走完（约 2 小时）

只看 5 段，不要贪。

① `run.py`（327 行，全读）

要搞明白：

- 启动顺序：起 uvicorn（后台线程）→ 轮询端口 → 开 pywebview 窗口 → 窗口关闭回收
- 开头那段 `if sys.stdout is None:` 是干嘛的（`pythonw` 没有控制台，任何 `print` 都会让进程直接死）

自测： 窗口关掉之后，后端进程会立刻退出吗？为什么？

---

② `frontend/index.html`（469 行，只看 `<body>`）

要搞明白：

- 界面分成几块：侧栏（会话/知识库/生成文库/图片库/定时任务）、顶栏（模型下拉、🔑 Key）、消息区、输入区
- `#modelSelect` 的 6 个 option 分别对应哪条通道（前 3 个本地 Ollama，后 3 个云端）

自测： "本地·自动"这一项，后端拿到的是字符串 `auto` 还是具体模型名？

---

③ `frontend/app.js`（6430 行）—— 本项目的"心脏"

看哪里：第 3093 行 `async function doSend(...)`，一路读到第 3900 行左右。

这条主线要跟下来：

```
用户点发送
  → send() 取输入、判空、判 streaming（第 3060 行附近，这里有个"卡住自愈"）
  → doSend() 建思考气泡 + 回答气泡
  → 判通道：_prov 是 deepseek* → /api/chat/deepseek，否则 /api/chat
  → reader.read() 循环：按 \n 切 NDJSON、逐行 JSON.parse
      · obj.message.content  → answer += ...  → paintAnswer()
      · obj.message.thinking → pushThinking()
      · obj.ping             → 只是心跳，重置看门狗
      · obj.__end            → answer = obj.answer（以服务端为准）+ 立刻重绘
  → 循环结束后：renderFinal(气泡, answer)   ← 第 1196 行
  → 回读服务端会话，比屏幕长就用它重绘
```

顺带看三个函数（都不长）：

行号	函数	作用
1182	`isPlainMode()`	纯文本 / markdown 的开关（默认 markdown，`?plain=1` 切纯文本）
1196	`renderFinal()`	统一渲染入口，**渲染后逐字核对结尾，丢了就退回纯文本**
1222	`addMsg()`	历史消息也走 `renderFinal`（早期只有实时消息走，导致"第二三条断"）

自测： 为什么流式阶段用 `textContent` 而不是 `mdToHtml`？（提示：想 mdToHtml 遇到未闭合的 `**` 会怎样）

---

④ `backend/main.py`（9133 行）—— 只看两个端点

行号	端点	读多少
4308	`@app.post("/api/chat")`	本地通道，从函数头读到 `gen()`（4775 行）结束
6552	`@app.post("/api/chat/deepseek")`	云端通道，重点看 `gen()`（6671 行）和它末尾的 `__end`

要搞明白：

- 为什么用 NDJSON（一行一个 JSON）而不是 SSE：前端按 `\n` 切，解析容错简单
- `__end` 里为什么带 `answer`（整段正文）：分片丢一个也有救
- 云端 `gen()` 末尾那段"服务端落盘"：为什么不依赖前端 PUT

自测： 如果前端在接收时丢了最后一片，靠哪两层能补回来？

---

⑤ `backend/deepseek_client.py`（746 行）

看哪里：第 355 行 `chat_stream_with_tools`，这是全项目最值得读的函数，只有 300 多行。

关键词：

- `content_buf` / `tool_calls`：流式下工具调用是**分片**的，按 `index` 累加
- `finish == "length"` → 自动续写（最多 6 次）
- `guard`：模型"光嘴上说说"（说要做文件却没调工具）时退回一轮

自测： 为什么 `finish_reason` 那一块要 `break` 出循环？它之前必须先 yield 什么？

第二遍 · 工具与安全（约 2 小时）

文件	行数	看什么
`backend/tools.py`	6436	从 `_do_` 开头的函数看：`_do_read_file`、`_do_run_python`、`_do_web_search`…每个都是"模型说一句 → 真正干活"的落点
`backend/selfdev.py`	900	自我迭代三件套：改前备份 → 改后语法校验 → 失败回滚；路径白名单怎么挡越界
`backend/keyvault.py`	94	最短最有代表性：定时任务半夜跑，没有浏览器，Key 从哪来（环境变量 > 本机保管）

自测： 模型要删一个文件，从它发出工具调用到真正 `os.remove`，中间经过几道校验？

第三遍 · 定时任务与配置（约 1 小时）

文件	行数	看什么
`backend/scheduler.py`	584	`_loop`（20 秒轮询 `next_run`）、`_run_once`（真跑一次）、`run_now`（第 428 行，后台线程 + 立即返回）
`backend/sessions.py`	283	会话落盘；`session_save` 里"**只许增长**"的取长者逻辑
`backend/config.py`	380	所有默认值：`max_tokens`、`no_output_limit`、`max_tool_rounds`、`perf_profile`
`README.md`	696	当手册查，不通读

自测： 定时任务"到点没跑"，你会先看哪三个地方？

附录 A • 关键锚点速查

```
run.py:12           pythonw 无 stdout 的兜底
app.js:3060         send() 的 streaming 卡死自愈
app.js:3093         doSend() 起点
app.js:1182         isPlainMode()
app.js:1196         renderFinal()   ← 渲染 + 尾部核对
app.js:1222         addMsg()        ← 历史消息渲染
app.js:2265         renderHistory()
main.py:4308        /api/chat（本地）
main.py:4775        本地通道 gen()
main.py:6552        /api/chat/deepseek（云端）
main.py:6671        云端通道 gen()   ← __end 全文 + 服务端落盘
main.py:8786        PUT /api/sessions/{sid}  ← 只增不减
deepseek_client.py:355   chat_stream_with_tools()
scheduler.py:428     run_now()
keyvault.py:1        全文件
```

附录 B • 五层"不丢字"是怎么叠起来的

层	位置	作用
1	`deepseek_client` 的 `content_buf`	生成侧完整，不受网络影响
2	`main.py` 云端 `gen()` 的 `__end.answer`	前端拿到整段原文，不靠分片拼
3	同上的 `sessions.save_messages`	服务端自己落盘一份，独立于前端
4	`session_save` 的"只许增长"	前端丢片上传的短版本覆盖不掉长原文
5	`renderFinal` 的尾部核对 + 前端回读	屏幕与原文逐字比对，缺了自动退回纯文本

自测： 把第 3 层去掉，什么场景会出问题？（提示：前端 PUT 发生在渲染之后）

附录 C · 常见"看起来像 bug"其实不是

现象	其实是
回答的 `innerText` 比日志里的字数少几十	markdown 标记（`**`、反引号、`-` ）不占可见字符
改了前端但界面没变	WebView 缓存；现在脚本带 `?v=时间戳`，且顶栏有版本号可核对
日志里 `__end local=N server=N` 相等但屏幕短	渲染层；`renderFinal` 会核对尾部并退回纯文本
定时任务"没反应"	一次要 60~120 秒，且后台跑；看任务卡片的 `last_run`，不是看按钮

阅读节奏建议

```
Day 1  上午：run.py + index.html          （打底，1 小时）
Day 1  下午：app.js 的 doSend 主线          （最费脑，慢慢来）
Day 1  晚上：main.py 两个端点 + __end       （对照前端看，会豁然开朗）
Day 2  上午：deepseek_client 全文          （300 行，值得逐行）
Day 2  下午：tools.py 挑 5 个 _do_ 函数      （不用全读）
Day 3  上午：selfdev + keyvault + scheduler （三个小文件）
Day 3  下午：自己动手改一个小功能，跑通       （这一步才算"吃透"）
```

最后一条建议： 读完不要立刻合上，去改一个小东西——比如把"发送"按钮的文案改掉，或给某个工具加一行日志。**能改通，才算读懂。**